claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a7e807a82054c4b5a.jsonl`
- SHA-256: `52b0655ceafda60dd66b712f09fe2904966d4c93cddd8cce1d0a87fcb6528f6b`
- Source modified: `2026-07-08T06:12:31+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_5f13fc79-bad`
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`

Transcript

1. user (2026-07-08T06:04:07.733Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to
an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).
The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):
- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical
  to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the
  SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config
  + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed
  once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing,
  never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

Your review dimension: robustness-security.
Review the #521 diff for ROBUSTNESS + SECURITY defects. Check: output_name traversal safety on the worker path (does the worker re-validate the name, or trust the spec?); malformed/partial spec handling; storage config forwarding (output_root/backend) so the worker can actually read the spec + upload the reel; exception paths that could crash instead of returning a failed job;

a worker validation-fail (skip_validation=False) surfacing a useful message vs a generic rc.

2. assistant (2026-07-08T06:04:12.197Z)

I'll start by locating the actual diff and changed files. The paths show "undefined" so I need to find the real PR #525 merge in git history.

3. user (2026-07-08T06:04:15.087Z)

```
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)
44a2f47 feat(detectors): GPU-resident WASB NVDEC decode behind default-OFF decode_backend flag
(#507)
17d844b docs(runlog): alpha-tester "All 1 v 1" cloud-serving run ? timeout diagnosed +
downsample fix proven (#504)
bd5647b fix(worker): surface detector failure/timeout as failed, not silent 0-rally success
(#503)
07e13f7 docs: Alpha Tester Onboarding FAQ ? send-ready copy for the first invite wave (#499)
2a8a931 docs: record the owner's §1.1 browser CUJ PASS ? DoD #1 satisfied (rev 00015-kp9) (#498)
6ab5a71 docs(runlog): CP-rebuild P9 executed ? rev 00012/00014, battery green, trackers updated
(#496)
257f469 chore(ui): re-vendor the SPA snapshot at khelsutra 90d1d90 (CP-rebuild P8) + batch the
§7 tracker updates (#495)
f6864f1 feat(recovery): restart-safe remote jobs + read-path re-hydrate (CP-rebuild P3, #471
#485) (#494)
```

4. user (2026-07-08T06:04:16.610Z)

```
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521)
428ee44 address follow-up review on #169: correct factual error in item-3 investigation note
(fusion_compare.py, fusion_golden.py, ablation.py + tests *do* exist per ls/grep on the tree;
person features in fusion_hybrid + detectors/fusion_features.py; remaining M0 in-container task
is wiring into compare). Cleaned duplicate sequencing paragraph. Verified files present. Plan
now accurate. M0 item 3 placement remains correct and even stronger.
318f6c2 test: positive tests for Item 2 (config) - shallow section replacement, unknown keys
warning, get shim, roundtrip + living doc update for T2 post #157
8836dfe chore: scripts/ for run-drivers + CODE_MAP §0 file-placement guidelines (#138)
```

5. assistant (2026-07-08T06:04:24.101Z)

The PR #521/#525 is merged. Let me examine the actual diff and the changed files.

6. user (2026-07-08T06:04:25.964Z)

```
commit 8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Wed Jul 8 11:19:30 2026 +0530
```

```
    perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement
(#521)

    The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the
    2026-07-08 live CUJ (rev rally-control-plane-00019, §9 of the validation-latency
    runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source
    (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode
    with no NVENC). Validation (183s) also runs there; both serialize on
    _LOCAL_COMPUTE_LANE, so one run pins the control-plane ~16 min ? won't scale past a
    handful of concurrent users.

    - Config-driven placement: deployment.phase_placement {validation|segment|compile}
      ? control_plane | cloud_run_l4, resolved by backend/config/placement.py. Unset ?
      today's placement, byte-identical. A config-load guard rejects the "nobody
      validates" footgun (validation on L4 without a cloud segment).
    - compile ? L4: a kind='compile' Cloud Run dispatch mirroring _maybe_dispatch_remote.
      The control-plane resolves the reel's segments, stages a compile-spec (resolved
      time-pairs + stitching config + source uri) to the bucket, dispatches
      `worker compile` on the L4, bucket-polls a done-marker, and returns the same
      {status, filepath, download_url} shape the SPA expects. The worker stitches via the
      SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.
    - NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default
      h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to
      libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob
      now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
    - validation delegation: phase_placement.validation moves the validator suite off the
      control-plane via the existing --skip-validation lever (#513), guarded so an
      in-process run always validates.
    - cloud-serving preset places compile on the L4.

    Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).
    Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to
    test_compute_backend.py (run_compile), test_worker.py (cmd_compile),
    test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation
    delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

    Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

 backend/compute/__init__.py                        |   2 +
 backend/compute/base.py                            |  34 +++
 backend/compute/cloud_run.py                       | 100 ++++++-
 backend/config/__init__.py                         |  16 ++
 backend/config/models.py                           |  52 ++++
 backend/config/placement.py                        |  59 ++++
 backend/config/presets.py                          |  10 +-
 backend/orchestration.py                           | 322 ++++++++++++++++++---
 backend/pipeline/stitcher/compiler.py              |  94 ++++++-
 backend/worker/__main__.py                         | 139 +++++++++
 docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md  | 169 ++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/README.md           |   1 +
 tests/test_api_cloud_dispatch.py                   |  70 +++++
 tests/test_compile_dispatch.py                     | 204 +++++++++++++
 tests/test_compiler.py                             |  64 +++++
 tests/test_compute_backend.py                      | 111 ++++++++
 tests/test_config_deployment.py                    |   5 +
 tests/test_phase_placement.py                      | 138 +++++++++
 tests/test_worker.py                               | 175 +++++++++++
```

7. user (2026-07-08T06:04:26.260Z)

```
 backend/compute/__init__.py                        |   2 +
 backend/compute/base.py                            |  34 +++
 backend/compute/cloud_run.py                       | 100 ++++++-
 backend/config/__init__.py                         |  16 ++
 backend/config/models.py                           |  52 ++++
```

```
 backend/config/placement.py                        |  59 ++++
 backend/config/presets.py                          |  10 +-
 backend/orchestration.py                           | 322 ++++++++++++++++++---
 backend/pipeline/stitcher/compiler.py              |  94 ++++++-
 backend/worker/__main__.py                         | 139 ++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md  | 169 ++++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/README.md           |   1 +
 tests/test_api_cloud_dispatch.py                   |  70 +++++
 tests/test_compile_dispatch.py                     | 204 ++++++++++++++
 tests/test_compiler.py                             |  64 +++++
 tests/test_compute_backend.py                      | 111 ++++++++
 tests/test_config_deployment.py                    |   5 +
 tests/test_phase_placement.py                      | 138 ++++++++++
 tests/test_worker.py                               | 175 ++++++++++++
 19 files changed, 1710 insertions(+), 55 deletions(-)
```

8. assistant (2026-07-08T06:04:29.210Z)

Let me get the full diff of the core changed files. Let me start with the worker, orchestration, cloud_run, and compiler.

9. user (2026-07-08T06:04:30.146Z)

```
commit 8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Wed Jul 8 11:19:30 2026 +0530

    perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement
(#521)

    The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the
    2026-07-08 live CUJ (rev rally-control-plane-00019, §9 of the validation-latency
    runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source
    (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode
    with no NVENC). Validation (183s) also runs there; both serialize on
    _LOCAL_COMPUTE_LANE, so one run pins the control-plane ~16 min ? won't scale past a
    handful of concurrent users.

    - Config-driven placement: deployment.phase_placement {validation|segment|compile}
      ? control_plane | cloud_run_l4, resolved by backend/config/placement.py. Unset ?
      today's placement, byte-identical. A config-load guard rejects the "nobody
      validates" footgun (validation on L4 without a cloud segment).
    - compile ? L4: a kind='compile' Cloud Run dispatch mirroring _maybe_dispatch_remote.
      The control-plane resolves the reel's segments, stages a compile-spec (resolved
      time-pairs + stitching config + source uri) to the bucket, dispatches
      `worker compile` on the L4, bucket-polls a done-marker, and returns the same
      {status, filepath, download_url} shape the SPA expects. The worker stitches via the
      SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.
    - NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default
      h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to
      libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob
      now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
    - validation delegation: phase_placement.validation moves the validator suite off the
      control-plane via the existing --skip-validation lever (#513), guarded so an
      in-process run always validates.
    - cloud-serving preset places compile on the L4.

    Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).
    Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to
    test_compute_backend.py (run_compile), test_worker.py (cmd_compile),
    test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation
    delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

    Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
```

```
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 6d3420b..79db808 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -419,6 +419,108 @@ def cmd_infer(args: argparse.Namespace) -> int:
     return 0


+def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
+    """Fetch + parse the control-plane-staged compile-spec (#521): the resolved ``time_pairs``,
the
+    resolved ``stitching`` config, the source ``video_uri``, ``video_id``, ``output_name`` +
``locale``.
+    A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real dst)."""
+    local = storage.fetch(
+        spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
+    )
+    with open(local, encoding="utf-8") as f:
+        data = json.load(f)
+    if not isinstance(data, dict):
+        raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
+    return data
+
+
+def _write_compile_marker(
+    done_uri: str,
+    result: dict,
+    video_id: str,
+    returncode: int,
+    storage_cfg: dict,
+    scratch: str,
+) -> None:
+    """#521: write the compile done-marker (rc + status + the reel's download_url/reel_uri) so
the
+    dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ? never
raises."""
+    try:
+        marker = {
+            "video_id": video_id,
+            "returncode": returncode,
+            "status": result.get("status", "failed"),
+            "download_url": result.get("download_url", ""),
+            "reel_uri": result.get("reel_uri", ""),
+            "message": result.get("message", ""),
+        }
+        local = os.path.join(scratch, "_compile_done.json")
+        with open(local, "w", encoding="utf-8") as f:
+            json.dump(marker, f)
+        storage.put(local, done_uri, config=storage_cfg)
+        logger.info("[worker] wrote compile marker -> %s", done_uri)
+    except Exception as e:  # never fail a good stitch because the marker push hiccuped
+        logger.warning("[worker] could not write compile done-marker %s: %s", done_uri, e)
+
+
+def cmd_compile(args: argparse.Namespace) -> int:
+    """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The stitch
is the
+    2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip 4K
reel). Read
+    the control-plane-staged compile-spec (resolved time-pairs + stitching config + source
URI), stitch
+    the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so there
is no
+    forked stitch pipeline), upload the reel to the bucket, and write a done-marker for the
dispatcher's
```

```
+    bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
+    from backend import orchestration
+    from backend.config import StitchingConfig
+
+    config = load_config(args.config) if args.config else load_config()
+    _apply_overrides(config, args.set)
+    storage_cfg = config.storage.model_dump()
+
+    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
+    os.makedirs(scratch, exist_ok=True)
+    # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to the
bucket.
+    config.output.output_dir = scratch
+    # The dispatcher forwards storage.output_root via --set; --out-uri carries the same value
as a
+    # fallback so reel_object_uri (which reads storage.output_root) always resolves the reel's
bucket
+    # destination even if the override were dropped.
+    if not (config.storage.output_root or "").strip():
+        config.storage.output_root = args.out_uri
+
+    spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
+    video_id = str(spec.get("video_id") or "")
+    out_name = str(spec.get("output_name") or "")
+    source_ref = str(spec.get("video_uri") or "")
+    locale = spec.get("locale")
+    time_pairs = [
+        (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
+    ]
+    stitching = StitchingConfig(**(spec.get("stitching") or {}))
+
+    print(
+        f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
+        f"encoder={stitching.encoder} out={out_name}"
+    )
+
+    result = orchestration._stitch_reel(
+        config,
+        stitching,
+        video_id,
+        source_ref,
+        time_pairs,
+        out_name,
+        locale,
+        lambda stage: print(f"[worker] compile: {stage}"),
+    )
+    status = str(result.get("status") or "failed")
+    rc = 0 if status == "success" else 3
+    if status != "success":
+        print(f"[worker] compile FAILED: {result.get('message')}")
+
+    # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher passes
it.
+    if getattr(args, "done_uri", None):
+        _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
+    return rc
+
+
 #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter so a
 #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem collision).
 _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
@@ -760,6 +862,43 @@ def build_parser() -> argparse.ArgumentParser:
     )
     pi.set_defaults(func=cmd_infer)
```

```
+    pc = sub.add_parser(
+        "compile",
+        help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged compile-
spec",
+    )
+    pc.add_argument(
+        "--spec-uri",
+        required=True,
+        help="storage URI of the control-plane-staged compile-spec JSON (time-pairs + stitching
"
+        "config + source uri + video_id)",
+    )
+    pc.add_argument(
+        "--out-uri",
+        default="",
+        help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
+        "Also forwarded via --set storage.output_root; either resolves the reel path.",
+    )
+    pc.add_argument(
+        "--done-uri",
+        default=None,
+        help="storage URI for the compile completion MARKER (rc + download_url), so a remote "
+        "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI stitch.",
+    )
+    pc.add_argument(
+        "--config", default=None, help="config.json path (default: ./config.json)"
+    )
+    pc.add_argument(
+        "--scratch", default=None, help="local scratch dir (default: a temp dir)"
+    )
+    pc.add_argument(
+        "--set",
+        action="append",
+        default=[],
+        metavar="k=v",
+        help="config override, e.g. --set storage.output_root=gs://bucket",
+    )
+    pc.set_defaults(func=cmd_compile)
+
     pt = sub.add_parser(
         "train",
         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
```

10. assistant (2026-07-08T06:04:32.559Z)

Now the orchestration.py diff ? this is the largest and most important.

11. user (2026-07-08T06:04:33.984Z)

```
commit 8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Wed Jul 8 11:19:30 2026 +0530

    perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement
(#521)

    The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the
    2026-07-08 live CUJ (rev rally-control-plane-00019, §9 of the validation-latency
    runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source
    (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode
    with no NVENC). Validation (183s) also runs there; both serialize on
    _LOCAL_COMPUTE_LANE, so one run pins the control-plane ~16 min ? won't scale past a
    handful of concurrent users.
```

```
    - Config-driven placement: deployment.phase_placement {validation|segment|compile}
      ? control_plane | cloud_run_l4, resolved by backend/config/placement.py. Unset ?
      today's placement, byte-identical. A config-load guard rejects the "nobody
      validates" footgun (validation on L4 without a cloud segment).
    - compile ? L4: a kind='compile' Cloud Run dispatch mirroring _maybe_dispatch_remote.
      The control-plane resolves the reel's segments, stages a compile-spec (resolved
      time-pairs + stitching config + source uri) to the bucket, dispatches
      `worker compile` on the L4, bucket-polls a done-marker, and returns the same
      {status, filepath, download_url} shape the SPA expects. The worker stitches via the
      SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.
    - NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default
      h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to
      libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob
      now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
    - validation delegation: phase_placement.validation moves the validator suite off the
      control-plane via the existing --skip-validation lever (#513), guarded so an
      in-process run always validates.
    - cloud-serving preset places compile on the L4.

    Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).
    Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to
    test_compute_backend.py (run_compile), test_worker.py (cmd_compile),
    test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation
    delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

    Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

diff --git a/backend/orchestration.py b/backend/orchestration.py
index 8b7851e..46ffaa4 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -26,11 +26,20 @@ import os
 import re
 import tempfile
 import threading
+import uuid
 from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
 from urllib.parse import quote

 from backend import storage
-from backend.config import AppConfig, StitchingConfig
+from backend.config import (
+    PHASE_COMPILE,
+    PHASE_SEGMENT,
+    PHASE_VALIDATION,
+    PLACE_CLOUD_L4,
+    AppConfig,
+    StitchingConfig,
+    resolve_phase_placement,
+)
 from backend.pipeline.segmenters.base import segmenter_registry
 from backend.pipeline.stitcher.watermark_i18n import localize_watermark
 from backend.pipeline.validators.base import ValidationResult, registry
@@ -206,6 +215,24 @@ def _cloud_available(cfg) -> bool:
     return bool(has_job and out_root)


+def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
+    """Would DETECT for this request run on the L4 worker (vs in-process)? Mirrors the
+    ``_maybe_dispatch_remote`` decision WITHOUT side effects (#521), so ``_execute_processing``
can
+    decide whether to DELEGATE validation to the worker (``phase_placement.validation``). A
per-request
+    picker choice wins; otherwise ``phase_placement.segment`` (whose legacy default is cloud
iff
```

```
+    ``compute_target==cloud_run``) gates it, and cloud must actually be available. A pre-
dispatch reject
+    (e.g. cloud requested but unconfigured) counts as NOT-on-cloud ? nothing runs remotely."""
+    choice = (req_compute or "").strip().lower()
+    if choice == "local":
+        return False
+    if choice == "cloud":
+        return _cloud_available(cfg)
+    return (
+        resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
+        and _cloud_available(cfg)
+    )
+
+
 def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
     """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
     minute-granular: the control plane knows ELAPSED time, not frames ? frame-level % comes
@@ -505,11 +532,19 @@ def _maybe_dispatch_remote(
             )
         compute = get_compute_backend(cfg, target_override="cloud_run")
     else:
-        compute = get_compute_backend(
-            cfg
-        )  # server default (default-OFF unless cloud_run)
+        # No explicit picker ? consult phase_placement.segment (#521). Its legacy default is
+        # cloud_run_l4 IFF compute_target==cloud_run, so this preserves today's behaviour;
setting
+        # phase_placement.segment relocates DETECT by config alone (e.g. force it back on the
+        # control-plane, or onto the L4 on a default-local box wired for cloud).
+        if (
+            resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
+            and _cloud_available(cfg)
+        ):
+            compute = get_compute_backend(cfg, target_override="cloud_run")
+        else:
+            compute = None
     if compute is None:
-        return None  # default-OFF ? run the in-process segmenter below (unchanged)
+        return None  # default-OFF / placed on control_plane ? the in-process segmenter below

     storage_cfg = getattr(cfg, "storage", None)

@@ -623,12 +658,20 @@ def _maybe_dispatch_remote(
     # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
     # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
     # then worker-rejected at its default 20.0 after a successful GPU dispatch).
+    # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the control-
plane is
+    # the authoritative gate ? it already validated with its resolved config (only a PASS
reaches
+    # here), so the worker SKIPS its redundant re-validation (#513). If validation is placed on
the
+    # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators ? see
+    # _execute_processing), so the worker must NOT skip: it becomes the gate.
+    worker_skips_validation = (
+        resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
+    )
     spec = ComputeJobSpec(
         video_uri=req.video_path,
         out_uri=out_root,
         segmenter=seg_name,
         config_overrides=tuple(overrides),
-        skip_validation=True,
+        skip_validation=worker_skips_validation,
```

```
+        skip_validation=worker_skips_validation,
     )
     try:
         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
@@ -811,12 +854,33 @@ def _execute_processing(
         # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors how it
bypasses
         # the whole-pipeline cache). Composes with #513 (the worker skips its own re-validation
on a
         # cloud dispatch), so a post-failure reupload reaches segmentation in ~0 validation
time.
-        val_fingerprint = registry.config_fingerprint(config)
+        #
+        # #521 phase_placement.validation: when validation is placed on the L4 AND detection
will
+        # actually dispatch there, the control-plane DELEGATES validation to the worker ? it
does NOT
+        # run the validator suite here (the whole point is to move the CPU-bound validation OFF
the
+        # 2-vCPU control-plane). The worker then validates (skip_validation=False) and rejects
a bad
+        # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed job. When
detection
+        # is NOT on the cloud (in-process), the control-plane always validates ? there is no
worker to
+        # delegate to (the config guard makes validation=cloud_run_l4 require a cloud segment).
+        validate_on_control_plane = (
+            resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
+            or not _will_detect_on_cloud(config, getattr(req, "compute", None))
+        )
+        if not validate_on_control_plane:
+            logger.info(
+                "[API] validation delegated to the L4 worker "
+                "(deployment.phase_placement.validation=cloud_run_l4) ? the control-plane skips
its "
+                "validator suite; the worker is the gate for %s.",
+                req.video_path,
+            )
+        val_fingerprint = (
+            registry.config_fingerprint(config) if validate_on_control_plane else ""
+        )
         # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault (unreadable
row,
         # corrupt or schema-incompatible stored results) degrades to a real validation rather
than
         # failing the job. The cache is a pure speedup ? never a new failure mode.
         reused_validation = False
-        if not req.force:
+        if validate_on_control_plane and not req.force:
             try:
                 cached = db.get_validation_cache(video_id)
                 if cached is not None and cached.get("fingerprint") == val_fingerprint:
@@ -841,11 +905,11 @@ def _execute_processing(
                     video_id,
                     exc_info=True,
                 )
-        if not reused_validation:
+        if validate_on_control_plane and not reused_validation:
             logger.info(f"Running validations for {req.video_path}...")
             val_results, val_context = registry.run_all_with_context(local_video, config)
         failures = [r for r in val_results if not r.passed]
-        if not reused_validation:
+        if validate_on_control_plane and not reused_validation:
             # Persist the verdict keyed by (content, config) so the NEXT identical reupload
skips the
```

```
            # decode. Best-effort (the writer swallows + logs its own faults); a miss just re-
validates.
            db.set_validation_cache(
@@ -1114,28 +1178,34 @@ def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
     return None


-def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
-    """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel,
-    and return the {status, filepath, download_url} result the SPA polls for. Returns a
-    ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a stitch
error)
-    so the SPA always gets a clean verdict instead of a worker-crash error."""
-    import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
+def _compile_wait_message(elapsed_sec: float) -> str:
+    """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
+    ``_wait_progress_message``). Minute-granular; under a minute reads as cold-start."""
+    minutes = int(elapsed_sec) // 60
+    if minutes < 1:
+        return "stitching on cloud GPU (NVENC) ? starting up"
+    return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"

-    notify = progress or (lambda stage: None)
-    params = json.loads(job.get("params") or "{}")
-    video_id = job.get("video_id") or ""
-    segment_ids = params.get("segment_ids") or []
-    locale = params.get("locale")

-    notify("resolving")
-    try:
-        video, kept, out_name = _resolve_compile(
-            db, cfg, video_id, segment_ids, params.get("output_filename", "")
-        )
-    except _CompileError as e:
-        return {"status": "failed", "message": e.detail, "video_id": video_id}
+def _stitch_reel(
+    cfg,
+    stitching,
+    video_id: str,
+    source_ref: str,
+    time_pairs: List[Tuple[float, float]],
+    out_name: str,
+    locale: Optional[str],
+    notify,
+) -> Dict[str, Any]:
+    """Stitch + mirror-upload core, shared by the control-plane compile handler AND the L4
worker
+    (#521). ``time_pairs`` are already RESOLVED (the caller filtered via ``_resolve_compile``),
so this
+    needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
+    with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source, runs
the
+    trim/watermark/concat, mirrors the reel to ``<output_root>/highlights/<video_id>/<name>``,
and
+    returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls for.
Returns a
+    ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
+    import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main

-    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
```

```
     output_dir = (
         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
     )
@@ -1147,8 +1217,8 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) ->
Dict[str, A
         begin_padding=stitching.begin_padding,
         end_padding=stitching.end_padding,
         watermark=localized_watermark,
+        encoder=getattr(stitching, "encoder", "libx264"),
     )
-    time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

     notify("stitching")
     local_src = ""
@@ -1157,7 +1227,7 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) ->
Dict[str, A
         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-scratch for
a URI (the
         # same seam the process path uses); without it, compiling a bucket-ingested video can't
open gs://.
         local_src = storage.acquire_local_input(
-            video["filepath"], getattr(cfg, "storage", None)
+            source_ref, getattr(cfg, "storage", None)
         )
         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count (for
remote
         # dispatch) doesn't run multiple compiles at once on the box (see _LOCAL_COMPUTE_LANE).
@@ -1175,13 +1245,14 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) ->
Dict[str, A
     finally:
         if local_src:
             storage.cleanup_acquired_local_input(
-                video["filepath"], local_src, getattr(cfg, "storage", None)
+                source_ref, local_src, getattr(cfg, "storage", None)
             )

     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed URL ?
Cloud Run
     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp copy dies
when the
     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch (the
endpoint
-    # still falls back to the local file on the same instance).
+    # still falls back to the local file on the same instance). On the L4 worker the mirror is
how the
+    # reel reaches the bucket at all (the worker's local file is discarded when the Job exits).
     reel_uri = reel_object_uri(cfg, video_id, out_name)
     if reel_uri:
         try:
@@ -1201,4 +1272,189 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) ->
Dict[str, A
         "filepath": out_path,
         "download_url": download_url,
         "video_id": video_id,
+        "reel_uri": reel_uri or "",
     }
+
+
+def _dispatch_compile_remote(
+    cfg,
+    db,
+    video_id: str,
+    segment_ids: List[int],
+    output_filename: str,
+    locale: Optional[str],
```

```
+    notify,
+) -> Dict[str, Any]:
+    """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-plane. The
+    control-plane owns the DB, so it RESOLVES the reel's segments here (``_resolve_compile``),
stages a
+    small compile-spec (resolved time-pairs + the resolved stitching/watermark config + the
source URI)
+    to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-polls the
+    done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
+    worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
+    ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
+    from backend.compute import CompileJobSpec, get_compute_backend
+
+    notify("resolving")
+    try:
+        video, kept, out_name = _resolve_compile(
+            db, cfg, video_id, segment_ids, output_filename
+        )
+    except _CompileError as e:
+        return {"status": "failed", "message": e.detail, "video_id": video_id}
+
+    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
+    storage_cfg = getattr(cfg, "storage", None)
+    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+    compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else None
+    if compute is None:
+        # Compile was placed on the L4 but the cloud dispatch isn't actually wired here (no
output
+        # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ? a slow
reel
+        # beats no reel. (The placement guard means this is only reachable in a mis-provisioned
setup.)
+        logger.warning(
+            "[compile] compile placed on the L4 but no cloud backend available (out_root=%r) ?
"
+            "stitching in-process on the control-plane instead.",
+            out_root,
+        )
+        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
+        return _stitch_reel(
+            cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale, notify
+        )
+
+    # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe on the
wire);
+    # the stitching config is the control-plane's RESOLVED one, with the encoder swapped to the
L4's
+    # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes the
reel.
+    stitch_dump = stitching.model_dump()
+    stitch_dump["encoder"] = getattr(
+        getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
+    )
+    spec_payload = {
+        "video_id": video_id,
+        "video_uri": video["filepath"],
+        "output_name": out_name,
+        "locale": locale,
+        "time_pairs": [
+            [float(s["start_time"]), float(s["end_time"])] for s in kept
+        ],
+        "stitching": stitch_dump,
+    }
+    token = uuid.uuid4().hex
```

```
+    spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
+    try:
+        notify("staging compile spec")
+        with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
+            local_spec = os.path.join(td, "spec.json")
+            with open(local_spec, "w", encoding="utf-8") as f:
+                json.dump(spec_payload, f)
+            storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
+    except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job failure, not a
crash
+        logger.error(
+            "[compile] failed to stage the compile spec to %s: %s", spec_uri, e, exc_info=True
+        )
+        return {
+            "status": "failed",
+            "message": f"cloud compile: could not stage the compile spec: {e}",
+            "video_id": video_id,
+        }
+
+    # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the spec,
mirror the
+    # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB knobs
for detect.
+    overrides = [f"storage.output_root={out_root}"]
+    backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
+    if backend_name:
+        overrides.append(f"storage.backend={backend_name}")
+    spec = CompileJobSpec(
+        out_uri=out_root,
+        spec_uri=spec_uri,
+        video_id=video_id,
+        output_name=out_name,
+        config_overrides=tuple(overrides),
+    )
+
+    notify("dispatching stitch (cloud_run)")
+    logger.info(
+        "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d clip(s))",
+        video_id,
+        out_root,
+        len(kept),
+    )
+    try:
+        result = compute.run_compile(
+            spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
+        )
+    except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+        logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
+        return {
+            "status": "failed",
+            "message": f"cloud compile dispatch error: {e}",
+            "video_id": video_id,
+        }
+    if result.returncode != 0:
+        msg = str(
+            result.report.get("message")
+            or f"cloud compile job failed (returncode {result.returncode})."
+        )
+        return {"status": "failed", "message": msg, "video_id": video_id}
+
+    download_url = str(
+        result.report.get("download_url")
+        or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
+    )
```

```
+    logger.info(
+        "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
+        result.execution,
+        result.outputs_uri,
+        video_id,
+    )
+    notify("finished")
+    return {
+        "status": "success",
+        "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
+        "download_url": download_url,
+        "video_id": video_id,
+        "compute": {
+            "path": "cloud_run",
+            "execution": result.execution,
+            "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
+            "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
+        },
+    }
+
+
+def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
+    """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the reel,
and return
+    the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
+    result for an EXPECTED problem (missing video / no segments / a stitch error) so the SPA
always gets
+    a clean verdict instead of a worker-crash error.
+
+    #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``) and cloud
+    dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
+    (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip reel on
the
+    2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process here,
unchanged."""
+    notify = progress or (lambda stage: None)
+    params = json.loads(job.get("params") or "{}")
+    video_id = job.get("video_id") or ""
+    segment_ids = params.get("segment_ids") or []
+    locale = params.get("locale")
+    output_filename = params.get("output_filename", "")
+
+    if (
+        resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
+        and _cloud_available(cfg)
+    ):
+        return _dispatch_compile_remote(
+            cfg, db, video_id, segment_ids, output_filename, locale, notify
+        )
+
+    notify("resolving")
+    try:
+        video, kept, out_name = _resolve_compile(
+            db, cfg, video_id, segment_ids, output_filename
+        )
+    except _CompileError as e:
+        return {"status": "failed", "message": e.detail, "video_id": video_id}
+
+    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
+    time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
+    return _stitch_reel(
+        cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale, notify
+    )
```

12. assistant (2026-07-08T06:04:36.875Z)

Now the compute layer (cloud_run.py, base.py) and compiler.py diffs.

13. user (2026-07-08T06:04:38.239Z)

```
+++ b/backend/compute/__init__.py
@@ -14,6 +14,7 @@ import os
 from typing import Any, Optional

 from backend.compute.base import (
+    CompileJobSpec,
     ComputeBackend,
     ComputeJobSpec,
     ComputeResult,
@@ -24,6 +25,7 @@ from backend.infrastructure.execution_policy import DEFAULT_POLICY,
ExecutionPol
 __all__ = [
     "ComputeBackend",
     "ComputeJobSpec",
+    "CompileJobSpec",
     "ComputeResult",
     "RemoteExecutionFailed",
     "get_compute_backend",
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 3ad76fc..f7afb40 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -51,6 +51,24 @@ class ComputeJobSpec:
     skip_validation: bool = False


+@dataclass(frozen=True)
+class CompileJobSpec:
+    """One reel-COMPILE job dispatched to the worker (#521). The heavy input ? the resolved
+    ``(start, end)`` time-pairs plus the control-plane's stitching/watermark config and the
source
+    URI ? is staged to ``spec_uri`` (a small JSON in the bucket) rather than crammed onto argv,
so
+    the worker needs no control-plane DB: it reads the spec, fetches the source, stitches on
the L4
+    (NVENC), uploads the reel to ``<out_uri>/highlights/<video_id>/<output_name>``, and writes
a
+    done-marker for the dispatcher's bucket-poll."""
+
+    out_uri: str  # output ROOT (gs://? bucket); highlights/<video_id>/<name> is written under
it
+    spec_uri: str  # staged compile-spec JSON (time-pairs + stitching + source uri + video_id)
+    video_id: str  # the reel's video_id (deterministic reel path + marker)
+    output_name: str  # traversal-safe reel filename
+    # Raw worker ``--set k=v`` overrides (storage.output_root/backend + stitching.encoder). A
tuple
+    # (not list/dict) so the spec stays frozen/hashable.
+    config_overrides: tuple[str, ...] = ()
+
+
 @dataclass(frozen=True)
 class ComputeResult:
     """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit code
@@ -85,3 +103,19 @@ class ComputeBackend(ABC):
         ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
         still WAITING on the remote work ? the caller's live-progress heartbeat (#482), so a
         long GPU run doesn't look frozen. Implementations without a wait loop may ignore it."""
+
+    def run_compile(
```

```
+        self,
+        spec: "CompileJobSpec",
+        *,
+        on_wait: "Callable[[float], None] | None" = None,
+    ) -> ComputeResult:
+        """Run a reel-COMPILE ``spec`` to completion and return its ``ComputeResult`` (#521).
+
+        Non-abstract with a NotImplementedError default: only backends that can offload the
ffmpeg
+        stitch (``CloudRunCompute``) override it ? an in-process/other backend never dispatches
a
+        compile. The returned ``ComputeResult.returncode`` is the worker's rc (0 ok / non-zero
=
+        stitch failure) and ``report`` is the worker's done-marker (carries the reel's
download_url)."""
+        raise NotImplementedError(
+            f"{type(self).__name__} does not support remote compile dispatch"
+        )
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index a4a0743..8df4246 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -37,6 +37,7 @@ from abc import ABC, abstractmethod
 from typing import Any, Callable, List, Optional

 from backend.compute.base import (
+    CompileJobSpec,
     ComputeBackend,
     ComputeJobSpec,
     ComputeResult,
@@ -278,21 +279,7 @@ class CloudRunCompute(ComputeBackend):
             done_uri,
         )

-        try:
-            marker = poll_until(
-                lambda: self._poll_check(done_uri, str(execution or "")),
-                self._policy,
-                label="cloud-run-infer",
-                logger=LOG,
-                # Live heartbeat (#482): each still-waiting tick reaches the caller so
-                # job.progress moves during the whole GPU run instead of freezing.
-                on_tick=on_wait,
-            )
-        except PolicyTimeout as timeout_exc:
-            # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
-            # so it propagates past here uncaught ? the dispatch handler maps it to a distinct,
-            # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
-            marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
+        marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-infer")
         video_id = str(marker.get("video_id", ""))
         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
         # garbled marker must not masquerade as rc=0. (The worker always writes both fields.)
@@ -317,6 +304,89 @@ class CloudRunCompute(ComputeBackend):
             execution=str(execution or ""),
         )

+    def run_compile(
+        self,
+        spec: "CompileJobSpec",
+        *,
+        on_wait: "Callable[[float], None] | None" = None,
```

```
+    ) -> ComputeResult:
+        """Dispatch a reel-COMPILE (#521): run ``python -m backend.worker compile`` on the L4
(NVENC),
+        bucket-poll the done-marker, and return the worker's rc + marker. Reuses the EXACT
+        dispatch?poll?marker machinery as ``run_infer`` (``_await_marker`` ? crash-fast +
timeout
+        rescue), so a stitch that crashes or overruns fails the same way a detection does ? not
a
+        ~65-min opaque hang."""
+        out_root = spec.out_uri.rstrip("/")
+        token = self._token or uuid.uuid4().hex
+        # Distinct marker prefix from infer's _dispatch/ so a compile + a detection for the
same
+        # bucket can never collide on a marker.
+        done_uri = f"{out_root}/_compile/{token}.done"
+        argv: List[str] = [
+            "compile",
+            "--out-uri",
+            spec.out_uri,
+            "--spec-uri",
+            spec.spec_uri,
+            "--done-uri",
+            done_uri,
+        ]
+        # Compile carries NO container-inline overrides: cmd_compile never consults
compute_target and
+        # never re-dispatches, so the only overrides are the CP-forwarded storage/encoder
knobs.
+        for kv in spec.config_overrides:
+            argv += ["--set", kv]
+
+        execution = self._client.run_job(
+            self._job_name, argv, region=self._region, project=self._project
+        )
+        LOG.info(
+            "cloud-run: dispatched COMPILE job=%s exec=%s; bucket-polling %s",
+            self._job_name,
+            execution,
+            done_uri,
+        )
+        marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-compile")
+        video_id = str(marker.get("video_id", "") or spec.video_id)
+        rc = int(marker.get("returncode", 1))
+        LOG.info(
+            "cloud-run: COMPILE job=%s done (video_id=%s rc=%d)",
+            self._job_name,
+            video_id,
+            rc,
+        )
+        # The marker carries the worker's stitch result (download_url / reel_uri / status);
surface it
+        # as ``report`` so the dispatcher returns the same {status, filepath, download_url}
shape the
+        # SPA expects. ``outputs_uri`` is the mirrored reel URI when present.
+        return ComputeResult(
+            returncode=rc,
+            outputs_uri=str(marker.get("reel_uri", "") or ""),
+            video_id=video_id,
+            report=marker,
+            execution=str(execution or ""),
+        )
+
+    def _await_marker(
+        self,
+        done_uri: str,
```

```
+        execution: object,
+        on_wait: "Callable[[float], None] | None",
+        *,
+        label: str,
+    ) -> dict:
+        """Bucket-poll ``done_uri`` to a completion marker ? shared by ``run_infer`` and
+        ``run_compile``. ``on_wait`` is the #482 live heartbeat (each still-waiting tick
reaches the
+        caller so job.progress moves). A ``RemoteExecutionFailed`` (the fast negative path in
+        ``_poll_check``) is NOT a ``PolicyTimeout`` so it propagates uncaught ? the dispatch
handler
+        maps it to a distinct failure; a genuine poll TIMEOUT is handled (rescue-or-cancel)."""
+        try:
+            return poll_until(
+                lambda: self._poll_check(done_uri, str(execution or "")),
+                self._policy,
+                label=label,
+                logger=LOG,
+                on_tick=on_wait,
+            )
+        except PolicyTimeout as timeout_exc:
+            return self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
+
     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
         """One bucket-poll iteration, hardened with an Executions-API fast negative path.
```

14. assistant (2026-07-08T06:04:40.671Z)

Now compiler.py, placement.py, and config models.

15. user (2026-07-08T06:04:42.517Z)

```
+++ b/backend/pipeline/stitcher/compiler.py
@@ -23,6 +23,49 @@ _BUNDLED_FONT = (
     Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
 )

+#: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's first clip
pays
+#: it and the rest reuse the verdict. Keyed by encoder name; cleared only across processes.
+_ENCODER_AVAILABLE: Dict[str, bool] = {}
+
+
+def _probe_encoder_available(encoder: str) -> bool:
+    """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny 1-frame
+    lavfi test encode to null. NVENC is often *compiled in* on boxes without an NVIDIA GPU, so
a
+    real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from "will fail
at
+    runtime". Memoized. Any error / timeout ? unavailable (caller falls back to libx264)."""
+    if encoder in _ENCODER_AVAILABLE:
+        return _ENCODER_AVAILABLE[encoder]
+    ok = False
+    # The probe is a tiny 1-frame encode ? bound it to a short timeout (? the configured ffmpeg
+    # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
+    configured = ffmpeg_timeout_sec()
+    probe_timeout = min(30.0, float(configured)) if configured else 30.0
+    try:
+        proc = subprocess.run(
+            [
+                ffmpeg_bin(),
+                "-hide_banner",
+                "-f",
+                "lavfi",
+                "-i",
```

```
+                "color=c=black:s=64x64:d=0.1",
+                "-frames:v",
+                "1",
+                "-c:v",
+                encoder,
+                "-f",
+                "null",
+                "-",
+            ],
+            capture_output=True,
+            timeout=probe_timeout,
+        )
+        ok = proc.returncode == 0
+    except Exception:  # noqa: BLE001 ? a probe failure/timeout just means "not usable ? fall
back"
+        ok = False
+    _ENCODER_AVAILABLE[encoder] = ok
+    return ok
+

 class VideoCompiler:
     def __init__(
@@ -31,6 +74,7 @@ class VideoCompiler:
         end_padding: float = 1.0,
         begin_padding: float = 0.5,
         watermark: Optional[Any] = None,
+        encoder: str = "libx264",
     ):
         self.output_dir = output_dir
         self.end_padding = end_padding
@@ -38,6 +82,11 @@ class VideoCompiler:
         # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
         # normalized to a dict (or None) so this module stays config-package-agnostic.
         self.watermark = self._normalize_watermark(watermark)
+        # Video encoder for the per-clip trim re-encode (#521). "libx264" (CPU) is the default
and
+        # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for the 4K
+        # re-encode the watermark forces. The requested encoder is PROBED once per run
+        # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run it.
+        self.encoder = (encoder or "libx264").strip() or "libx264"
         os.makedirs(output_dir, exist_ok=True)

     @staticmethod
@@ -225,6 +274,30 @@ class VideoCompiler:
             return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
         return drawtext

+    def _select_encoder(self) -> Tuple[List[str], str]:
+        """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run, plus
the
+        chosen encoder name. Chosen ONCE (not per clip) so all clips share one codec and the
final
+        concat can stay ``-c copy``. NVENC (hardware) is the L4 win on the 4K re-encode; if the
+        requested NVENC encoder can't run in this ffmpeg build (CI / non-GPU box), fall back to
+        libx264 so a misconfig never nukes the reel."""
+        enc = self.encoder
+        if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
+            logger.warning(
+                "[compile] encoder %r is not runnable in this ffmpeg build ? falling back to "
+                "libx264 (CPU). On the L4 worker this should be available.",
+                enc,
+            )
+            enc = "libx264"
+        if enc == "libx264":
+            # Unchanged historical CPU settings (byte-compatible with the pre-#521 stitch).
```

```
+            return ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"], enc
+        # NVENC (H.264/HEVC): hardware encode. -cq is the CRF-equivalent constant-quality knob;
p4 is
+        # a balanced speed/quality preset. yuv420p keeps the output broadly playable.
+        return (
+            ["-c:v", enc, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"],
+            enc,
+        )
+
     def _trim_one_segment(
         self,
         idx: int,
@@ -235,6 +308,7 @@ class VideoCompiler:
         tmp_dir: str,
         raw_video_path: str,
         watermark_filter: Optional[str],
+        video_codec_args: Optional[List[str]] = None,
     ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
         """Trim a single segment into its own clip file (extracted verbatim from the
         per-segment body of compile_highlights). Returns (clip_path, skip_record): on
@@ -297,6 +371,13 @@ class VideoCompiler:
             f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s] (duration:
{trim_duration:.2f}s)"
         )

+        # Default to libx264 (the historical settings) when a caller invokes _trim_one_segment
+        # directly without resolving an encoder ? keeps this method's old behaviour intact.
+        codec_args = (
+            video_codec_args
+            if video_codec_args is not None
+            else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
+        )
         base_cmd = [
             ffmpeg_bin(),
             "-y",
@@ -306,12 +387,7 @@ class VideoCompiler:
             str(round(padded_end, 3)),
             "-i",
             raw_video_path,
-            "-c:v",
-            "libx264",
-            "-preset",
-            "superfast",
-            "-crf",
-            "22",
+            *codec_args,
             "-c:a",
             "aac",
             "-b:a",
@@ -445,6 +521,11 @@ class VideoCompiler:
             watermark_filter = self._watermark_filter(tmp_dir)
             if watermark_filter:
                 logger.info("[watermark] applying branding overlay to each clip.")
+            # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip shares one
codec
+            # and the final concat stays -c copy (#521).
+            video_codec_args, chosen_encoder = self._select_encoder()
+            if chosen_encoder != "libx264":
+                logger.info("[compile] encoding clips with %s (hardware).", chosen_encoder)
             for idx, (raw_start, raw_end) in enumerate(segments):
                 clip_path, skip_record = self._trim_one_segment(
                     idx,
@@ -455,6 +536,7 @@ class VideoCompiler:
                     tmp_dir,
                     raw_video_path,
```

```
                    watermark_filter,
+                   video_codec_args,
                )
                if clip_path is not None:
                    temp_clips.append(clip_path)
```

16. user (2026-07-08T06:04:43.174Z)

```
+++ b/backend/config/__init__.py
@@ -19,10 +19,19 @@ from .models import (  # re-export common ones
     Config,
     DeploymentConfig,
     IndexingConfig,
+    PhasePlacementConfig,
     StitchingConfig,
     StorageConfig,
     ValidationConfig,
 )
+from .placement import (
+    PHASE_COMPILE,
+    PHASE_SEGMENT,
+    PHASE_VALIDATION,
+    PLACE_CLOUD_L4,
+    PLACE_CONTROL_PLANE,
+    resolve_phase_placement,
+)
 from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
 from .startup import (
     StartupCheck,
@@ -52,9 +61,16 @@ __all__ = [
     "Config",
     "DeploymentConfig",
     "IndexingConfig",
+    "PhasePlacementConfig",
     "StitchingConfig",
     "StorageConfig",
     "ValidationConfig",
+    "resolve_phase_placement",
+    "PHASE_VALIDATION",
+    "PHASE_SEGMENT",
+    "PHASE_COMPILE",
+    "PLACE_CONTROL_PLANE",
+    "PLACE_CLOUD_L4",
     "PROFILE_PRESETS",
     "suggest_profile",
     "probe_cuda_available",
diff --git a/backend/config/models.py b/backend/config/models.py
index a7c997c..ce8e98a 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -364,6 +364,14 @@ class StitchingConfig(BaseModel):
     # background-court fragments), so this keeps the reel to the eye-catching long rallies.
     # OFF by default ? the detector output is unchanged, only which rallies reach the reel.
     min_rally_duration: float = Field(default=0.0, ge=0.0)
+    # Video encoder for the per-clip trim re-encode (the branding watermark forces a re-encode;
the
+    # concat stays -c copy). Default ``libx264`` (CPU, works everywhere incl. CI).
``h264_nvenc`` /
+    # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-encode
that
+    # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested encoder
ONCE and
+    # falls back to libx264 if this ffmpeg build can't run it, so a non-GPU box never breaks.
The
+    # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for the L4
```

```
run;
+    # the control-plane's own local stitch keeps libx264.
+    encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


@@ -451,6 +459,21 @@ class StorageConfig(BaseModel):
     gdrive_api: dict[str, Any] = Field(default_factory=dict)


+class PhasePlacementConfig(BaseModel):
+    """Per-phase machine placement (#521): move validation / segment(detect) / compile(stitch)
+    between the control-plane and the L4 Cloud Run worker **by config, no code change**.
+
+    Each field is ``None`` (? the legacy default for that phase ? see
+    ``backend/config/placement.py``) or an explicit ``"control_plane"`` / ``"cloud_run_l4"``.
+    ``control_plane`` runs the phase in-process on the CPU-only control-plane; ``cloud_run_l4``
+    dispatches it to the GPU/NVENC worker Job. Unset ? byte-identical to pre-#521 behaviour.
+    Resolve via ``backend.config.resolve_phase_placement(cfg, phase)`` ? never read these
directly."""
+
+    validation: Optional[Literal["control_plane", "cloud_run_l4"]] = None
+    segment: Optional[Literal["control_plane", "cloud_run_l4"]] = None
+    compile: Optional[Literal["control_plane", "cloud_run_l4"]] = None
+
+
 class DeploymentConfig(BaseModel):
     """Deployment profile ? one switch that selects a coherent bundle of compute + storage
     knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
@@ -481,6 +504,35 @@ class DeploymentConfig(BaseModel):
     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect + queue/cold-
start,
     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid GPU run.
     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
+    # Per-phase machine placement (#521). Unset ? today's placement (validation + compile on
the
+    # control-plane, segment on the L4 iff compute_target==cloud_run). Set a phase to relocate
it by
+    # config alone. Resolve via ``backend.config.resolve_phase_placement`` (never read the sub-
fields
+    # directly ? the resolver folds in the legacy compute_target default).
+    phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)
+    # Video encoder the cloud compile-dispatch forwards onto ``stitching.encoder`` for the L4
stitch
+    # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the 4K re-encode the watermark
forces;
+    # set ``libx264`` to run the L4 stitch on CPU. Only consulted when compile is placed on the
L4;
+    # the control-plane's own local stitch always uses ``stitching.encoder`` (libx264 by
default).
+    cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
+
+    @model_validator(mode="after")
+    def _validation_placement_needs_cloud_segment(self) -> "DeploymentConfig":
+        # A clip can only be validated ON the worker if the worker also DETECTS it there ? the
worker
+        # is only launched for the segment(detect) phase. So validation=cloud_run_l4 with
segment on
+        # the control-plane would leave NOBODY validating (the control-plane skips, and no
worker runs)
+        # ? reject that footgun at config load rather than ship a silently-unvalidated clip.
+        if self.phase_placement.validation == "cloud_run_l4":
+            seg = self.phase_placement.segment or (
+                "cloud_run_l4" if self.compute_target == "cloud_run" else "control_plane"
+            )
```

```
+            if seg != "cloud_run_l4":
+                raise ValueError(
+                    "deployment.phase_placement.validation='cloud_run_l4' requires segment to
also "
+                    "run on the L4 ? the worker can only validate a clip it also detects. Set "
+                    "phase_placement.segment='cloud_run_l4' (or compute_target='cloud_run'), or
keep "
+                    "validation on the control_plane."
+                )
+        return self


 class BillingConfig(BaseModel):
diff --git a/backend/config/placement.py b/backend/config/placement.py
new file mode 100644
index 0000000..cda3ca5
--- /dev/null
+++ b/backend/config/placement.py
@@ -0,0 +1,59 @@
+"""Phase ? machine placement resolution (#521).
+
+A *phase placement* answers one question per pipeline phase ? **where does the heavy compute
for
+this phase run?** ? with a config knob (``deployment.phase_placement``) instead of code. The
three
+placeable phases and their values:
+
+    validation  ?  control_plane | cloud_run_l4
+    segment     ?  control_plane | cloud_run_l4   (a.k.a. "detect")
+    compile     ?  control_plane | cloud_run_l4   (a.k.a. "stitch")
+
+``control_plane`` runs the phase IN-PROCESS on the (CPU-only) control-plane service;
``cloud_run_l4``
+dispatches it to the GPU/NVENC Cloud Run worker Job (the same L4 detection already uses). This
module
+is the SINGLE source of truth so ``backend.orchestration`` (segment + compile) and the worker
never
+re-derive the mapping.
+
+**Default = today's behaviour (pure / additive).** When a phase's knob is unset (``None``) the
+resolver reproduces the pre-#521 placement:
+
+    validation  ?  control_plane                                    (the authoritative gate,
#512/#513)
+    segment     ?  cloud_run_l4 IFF deployment.compute_target == "cloud_run", else
control_plane
+    compile     ?  control_plane                                    (the in-process ffmpeg
stitch)
+
+so a config with no ``phase_placement`` block behaves byte-identically to before this module
existed.
+"""
+
+from __future__ import annotations
+
+from typing import Any
+
+# Phase identities (kept as constants so callers don't stringly-type the phase names).
+PHASE_VALIDATION = "validation"
+PHASE_SEGMENT = "segment"
+PHASE_COMPILE = "compile"
+PHASES: tuple[str, ...] = (PHASE_VALIDATION, PHASE_SEGMENT, PHASE_COMPILE)
+
+# Placement targets.
+PLACE_CONTROL_PLANE = "control_plane"
```

```
+PLACE_CLOUD_L4 = "cloud_run_l4"
+PLACEMENTS: tuple[str, ...] = (PLACE_CONTROL_PLANE, PLACE_CLOUD_L4)
+
+
+def resolve_phase_placement(cfg: Any, phase: str) -> str:
+    """Where does ``phase`` run ? ``"control_plane"`` or ``"cloud_run_l4"``?
+
+    Precedence: an explicit ``deployment.phase_placement.<phase>`` wins; otherwise the legacy
default
+    (mirrors the pre-#521 behaviour, see the module docstring). Never raises ? an absent
+    ``deployment`` / ``phase_placement`` degrades to the legacy default.
+    """
+    deployment = getattr(cfg, "deployment", None)
+    placement = getattr(deployment, "phase_placement", None)
+    explicit = getattr(placement, phase, None) if placement is not None else None
+    if explicit:
+        return str(explicit)
+    # Legacy defaults ? reproduce today's placement exactly.
+    if phase == PHASE_SEGMENT:
+        compute_target = (getattr(deployment, "compute_target", "") or "").strip()
+        return PLACE_CLOUD_L4 if compute_target == "cloud_run" else PLACE_CONTROL_PLANE
+    # validation + compile default to the control-plane (their pre-#521 home).
+    return PLACE_CONTROL_PLANE
```

17. assistant (2026-07-08T06:04:48.528Z)

Let me read the full post-change source around key functions to understand traversal safety and how `out_name`/`reel_object_uri` work.

18. user (2026-07-08T06:04:51.247Z)

```
backend/orchestration.py:5:and the shared compile resolution + worker handler
(``_resolve_compile`` / ``_execute_compile``).
backend/orchestration.py:38:from backend.utils.paths import output_base, safe_output_filename
backend/orchestration.py:1036:def _resolve_compile(
backend/orchestration.py:1037:    db, cfg, video_id: str, segment_ids: List[int],
output_filename: str
backend/orchestration.py:1042:    so manual/legacy picks can't silently vanish), and the output
name is traversal-safe. Raises
backend/orchestration.py:1043:    ``_CompileError(404/400)``. Returns ``(video_row,
kept_segments, safe_out_name)``."""
backend/orchestration.py:1047:    # Reject path traversal / absolute names (os.path.join would
otherwise write anywhere on disk).
backend/orchestration.py:1049:        out_name = safe_output_filename(output_filename)
backend/orchestration.py:1082:    return video, kept, out_name
backend/orchestration.py:1085:def reel_object_uri(cfg, video_id: str, name: str) -> str:
backend/orchestration.py:1121:    reel_uri = reel_object_uri(cfg, video_id, name)
backend/orchestration.py:1152:        video, kept, out_name = _resolve_compile(
backend/orchestration.py:1153:            db, cfg, video_id, segment_ids,
params.get("output_filename", "")
backend/orchestration.py:1180:        out_name,
backend/orchestration.py:1193:            out_path = compiler.compile_highlights(local_src,
time_pairs, out_name)
backend/orchestration.py:1213:    reel_uri = reel_object_uri(cfg, video_id, out_name)
backend/orchestration.py:1225:    download_url =
f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
backend/orchestration.py:1232:        "filename": out_name,
backend/pipeline/stitcher/compiler.py:444:        output_filename: str,
backend/pipeline/stitcher/compiler.py:481:        output_path = os.path.join(self.output_dir,
os.path.basename(output_filename))
```

19. user (2026-07-08T06:04:56.829Z)

```
1036    def _resolve_compile(
```

```
        db, cfg, video_id: str, segment_ids: List[int], output_filename: str
    ):
        """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-resolve, since a
        row could change between submit and run). Verifies the video exists, the requested segments match
        and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known shot_count are exempt
        so manual/legacy picks can't silently vanish), and the output name is traversal-safe. Raises
        ``_CompileError(404/400)``. Returns ``(video_row, kept_segments, safe_out_name)``."""
        video = db.get_video(video_id)
        if not video:
            raise _CompileError(404, "Indexed video record not found.")
        # Reject path traversal / absolute names (os.path.join would otherwise write anywhere on disk).
        try:
            out_name = safe_output_filename(output_filename)
        except ValueError as e:
            raise _CompileError(400, str(e)) from e

        all_segments = db.query_play_segments(video_id, {})
        matched = [s for s in all_segments if s["id"] in segment_ids]
        if not matched:
            raise _CompileError(400, "No matching play segments found to stitch.")

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched):
            logger.info(
                "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the floor.",
                stitching.min_shot_count,
                len(matched) - len(kept),
                len(matched),
            )
        if not kept:
            raise _CompileError(
                400,
                f"All {len(matched)} matching segments fall below "
                f"stitching.min_shot_count={stitching.min_shot_count}.",
            )
        return video, kept, out_name


    def reel_object_uri(cfg, video_id: str, name: str) -> str:
        """The bucket URI a compiled reel is mirrored to for a direct signed-URL download (#463), or "" when
        the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the compile
        mirror-upload and the /api/highlights redirect: ``<output_root>/highlights/<video_id>/<name>``."""
        root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
        if not root or "://" not in root:
            return ""
```

```
        return f"{root.rstrip('/')}/highlights/{video_id}/{name}"


    def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
        """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
        GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
        isn't silently constructed as a fresh default."""
        sc = getattr(cfg, "storage", None)
        return sc.model_dump() if sc is not None else None


    def _rendition_encode_profile(stitching: Any, rendition: str) -> Optional[Any]:
        if rendition != "sd":
            return None
        renditions = (
            stitching.get("renditions")
            if isinstance(stitching, dict)
            else getattr(stitching, "renditions", None)
        )
        if isinstance(renditions, dict):
            return renditions.get("sd")
        return getattr(renditions, "sd", None)


    def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
        """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
        307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
        /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
        the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
        reel_uri = reel_object_uri(cfg, video_id, name)
        if not reel_uri:
            return None
        settings = _storage_settings(cfg)
        if storage.exists(reel_uri, config=settings):
            return storage.signed_url(reel_uri, config=settings, method="GET")
        return None


    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
        """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
        and return the {status, filepath, download_url} result the SPA polls for. Returns a
        ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
        so the SPA always gets a clean verdict instead of a worker-crash error."""
        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main

        notify = progress or (lambda stage: None)
        params = json.loads(job.get("params") or "{}")
        video_id = job.get("video_id") or ""
        segment_ids = params.get("segment_ids") or []
        locale = params.get("locale")
        rendition = str(params.get("rendition") or "original")
        if rendition not in {"original", "sd"}:
            return {
                "status": "failed",
                "message": f"Unsupported compile rendition: {rendition}",
                "video_id": video_id,
```

```
            }

        notify("resolving")
        try:
            video, kept, out_name = _resolve_compile(
                db, cfg, video_id, segment_ids, params.get("output_filename", "")
            )
        except _CompileError as e:
            return {"status": "failed", "message": e.detail, "video_id": video_id}

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        output_dir = (
            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
        )
        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
        # unknown/absent locale keeps the configured default unchanged.
        localized_watermark = localize_watermark(stitching.watermark, locale)
        encode_profile = _rendition_encode_profile(stitching, rendition)
```

20. user (2026-07-08T06:04:57.437Z)

```
      """Path-safety helpers for the API boundary (untrusted request fields)."""

      import os
      import re
      from typing import Any, Optional


      def output_base(config: Any) -> str:
          """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
          (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
          chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
          a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.

          This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
          ``gs://`` URI you can't ``makedirs``). Accepts the typed AppConfig or the legacy
dict-shim
          (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``"output"``."""
          try:
              out = (
                  config.output.output_dir
                  if not isinstance(config, dict)
                  else config.get("output", {}).get("output_dir")
              )
          except Exception:
              out = None
          return out or "output"


      def safe_output_filename(name: str) -> str:
          """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).

          `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
          and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
          primitive. Returns the name unchanged when safe; raises ValueError otherwise.
          """
```

```
35          if not name or not name.strip():
36              raise ValueError("output_filename must not be empty")
37          if "\x00" in name:
38              raise ValueError("output_filename must not contain a null byte")
39          if "/" in name or "\\" in name:
40              raise ValueError(f"output_filename must not contain a path separator: {name!r}")
41          if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
42              raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
43          return name
44
45
46      def resolve_under_root(path: str, root: str) -> str:
47          """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
48          rp = os.path.realpath(path)
49          rr = os.path.realpath(root)
50          try:
51              contained = os.path.commonpath([rp, rr]) == rr
52          except ValueError:  # different drives on Windows
53              contained = False
54          if not contained:
55              raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
56          return rp
57
58
59      # A storage URI (gs://, s3://, gdrive://, ?) starts with a scheme; it is NOT a local
filesystem path,
60      # so os.path.realpath() would mangle it into a bogus cwd-relative path ? the local-path
containment
61      # jail can't be applied to it. A bucket URI is instead scoped to the configured
input_root (below).
62      _URI_SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
63
64
65      def _uri_under_root(uri: str, root: str) -> bool:
66          """True if the storage URI ``uri`` is exactly ``root`` or a child of it ? a
normalized-prefix check
67          with a separator boundary, so ``gs://b/videos-evil`` is NOT under
``gs://b/videos``."""
68          u = str(uri).rstrip("/")
69          r = str(root).rstrip("/")
70          return bool(r) and (u == r or u.startswith(r + "/"))
71
72
73      def validate_input_path(
74          path: str,
75          allowed_root: Optional[str] = None,
76          *,
77          allowed_uri_root: Optional[str] = None,
78      ) -> None:
79          """Reject null bytes and (in hosted mode) out-of-bounds inputs.
80
81          With no ``allowed_root`` (default, single-user local mode) only the null-byte check
applies, so the
82          local 'point it at any of my videos' workflow is preserved.
83
84          Setting ``allowed_root`` (a hosted/tenant deployment) enforces containment so an
authenticated
85          tenant can't point the engine at data it shouldn't reach:
86            - a LOCAL path must resolve under ``allowed_root`` (a realpath containment jail);
and
87            - a storage URI (gs://, s3://, gdrive://) ? which realpath can't jail ? is allowed
ONLY when it is
88              under ``allowed_uri_root`` (the configured ``storage.input_root`` bucket
prefix). An arbitrary
```

```
89              bucket, or any URI when no ``allowed_uri_root`` is configured, is REJECTED
(tenant isolation).
90
91          The null-byte check always applies.
92          """
93          if "\x00" in path:
94              raise ValueError("video_path must not contain a null byte")
95          if not allowed_root:
96              return
97          if _URI_SCHEME_RE.match(path):
98              if _uri_under_root(path, allowed_uri_root or ""):
99                  return
100             raise ValueError(
101                 f"hosted mode: {path!r} is not under the allowed input root "
102                 f"{(allowed_uri_root or '')!r}"
103             )
104         resolve_under_root(path, allowed_root)
105
```

21. assistant (2026-07-08T06:05:44.879Z)

Let me read the full `_stitch_reel` and the worker `cmd_compile` post-change, plus the worker's top-level dispatch to see how exceptions/markers are handled.

22. user (2026-07-08T06:05:46.095Z)

```
1180            out_name,
1181        )
1182        local_src = ""
1183        try:
1184            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1185            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1186            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1187            local_src = storage.acquire_local_input(
1188                video["filepath"], getattr(cfg, "storage", None)
1189            )
1190            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1191            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1192            with _LOCAL_COMPUTE_LANE:
1193                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1194        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1195            logger.error(
1196                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1197            )
1198            return {
1199                "status": "failed",
1200                "message": f"Highlight stitching failed: {e}",
1201                "video_id": video_id,
1202            }
1203        finally:
1204            if local_src:
1205                storage.cleanup_acquired_local_input(
1206                    video["filepath"], local_src, getattr(cfg, "storage", None)
1207                )
1208
1209        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1210        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
```

```
1211        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1212        # still falls back to the local file on the same instance).
1213        reel_uri = reel_object_uri(cfg, video_id, out_name)
1214        if reel_uri:
1215            try:
1216                notify("uploading reel")
1217                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1218                # project/client) is used, not a fresh default.
1219                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1220            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1221                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1222
1223        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1224        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1225        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1226        notify("finished")
1227        return {
1228            "status": "success",
1229            "filepath": out_path,
1230            "download_url": download_url,
1231            "video_id": video_id,
1232            "filename": out_name,
1233            "rendition": rendition,
1234        }
1235
```

23. assistant (2026-07-08T06:05:59.703Z)

The current file differs from commit 8866ffe (it has `rendition` handling not in that diff). Let me verify which version is actually on master and reconcile.

24. user (2026-07-08T06:06:04.280Z)

```
=== HEAD ===
bf97700facc2c7762a4cb2c7c039e7673eebab8b
=== is dac6736 (merge #525) ancestor of HEAD? ===
NO dac6736 NOT in master
=== is 8866ffe ancestor of HEAD? ===
NO 8866ffe NOT in master
```

25. user (2026-07-08T06:06:05.961Z)

```
=== branches containing 8866ffe ===
+ feat/521-compile-on-l4
  remotes/origin/feat/521-compile-on-l4
=== log --oneline first-parent top 8 ===
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
```

26. assistant (2026-07-08T06:06:14.480Z)

```
The PR is on branch `feat/521-compile-on-l4` (commit 8866ffe), NOT yet in the master working
tree. I must review the branch version, not the working tree. Let me extract the post-change
files from that commit.
```

27. user (2026-07-08T06:06:19.918Z)

```
written
1190:def _stitch_reel(
```

28. user (2026-07-08T06:06:24.638Z)

```
1190    def _stitch_reel(
1191        cfg,
1192        stitching,
1193        video_id: str,
1194        source_ref: str,
1195        time_pairs: List[Tuple[float, float]],
1196        out_name: str,
1197        locale: Optional[str],
1198        notify,
1199    ) -> Dict[str, Any]:
1200        """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1201        (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1202        needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1203        with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1204        trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1205        returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1206        ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
1207        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
1208
1209        output_dir = (
1210            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1211        )
1212        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1213        # unknown/absent locale keeps the configured default unchanged.
1214        localized_watermark = localize_watermark(stitching.watermark, locale)
1215        compiler = _main.VideoCompiler(
1216            output_dir,
1217            begin_padding=stitching.begin_padding,
1218            end_padding=stitching.end_padding,
1219            watermark=localized_watermark,
1220            encoder=getattr(stitching, "encoder", "libx264"),
1221        )
1222
1223        notify("stitching")
1224        local_src = ""
1225        try:
1226            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1227            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1228            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1229            local_src = storage.acquire_local_input(
1230                source_ref, getattr(cfg, "storage", None)
1231            )
```

```
            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
            with _LOCAL_COMPUTE_LANE:
                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
            logger.error(
                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
            )
            return {
                "status": "failed",
                "message": f"Highlight stitching failed: {e}",
                "video_id": video_id,
            }
        finally:
            if local_src:
                storage.cleanup_acquired_local_input(
                    source_ref, local_src, getattr(cfg, "storage", None)
                )

        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
        # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
        # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
        reel_uri = reel_object_uri(cfg, video_id, out_name)
        if reel_uri:
            try:
                notify("uploading reel")
                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
                # project/client) is used, not a fresh default.
                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)

        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
        notify("finished")
        return {
            "status": "success",
            "filepath": out_path,
            "download_url": download_url,
            "video_id": video_id,
            "reel_uri": reel_uri or "",
        }


    def _dispatch_compile_remote(
        cfg,
        db,
        video_id: str,
        segment_ids: List[int],
        output_filename: str,
        locale: Optional[str],
```

```
1286        notify,
1287    ) -> Dict[str, Any]:
1288        """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
1289        control-plane owns the DB, so it RESOLVES the reel's segments here
(``_resolve_compile``), stages a
```

29. assistant (2026-07-08T06:06:28.370Z)

Now let me read the full worker `cmd_compile` and the worker main dispatch (how `func` exceptions are handled and whether a failure marker gets written on crash).

30. user (2026-07-08T06:06:31.393Z)

```
419         return 0
420
421
422     def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
423         """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
424         resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
425         A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
426         local = storage.fetch(
427             spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
428         )
429         with open(local, encoding="utf-8") as f:
430             data = json.load(f)
431         if not isinstance(data, dict):
432             raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
433         return data
434
435
436     def _write_compile_marker(
437         done_uri: str,
438         result: dict,
439         video_id: str,
440         returncode: int,
441         storage_cfg: dict,
442         scratch: str,
443     ) -> None:
444         """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
445         dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
446         try:
447             marker = {
448                 "video_id": video_id,
449                 "returncode": returncode,
450                 "status": result.get("status", "failed"),
451                 "download_url": result.get("download_url", ""),
452                 "reel_uri": result.get("reel_uri", ""),
453                 "message": result.get("message", ""),
454             }
455             local = os.path.join(scratch, "_compile_done.json")
456             with open(local, "w", encoding="utf-8") as f:
457                 json.dump(marker, f)
458             storage.put(local, done_uri, config=storage_cfg)
459             logger.info("[worker] wrote compile marker -> %s", done_uri)
460         except Exception as e:  # never fail a good stitch because the marker push hiccuped
461             logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)
462
463
```

```
    def cmd_compile(args: argparse.Namespace) -> int:
        """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The stitch is the
        2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip 4K reel). Read
        the control-plane-staged compile-spec (resolved time-pairs + stitching config + source URI), stitch
        the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so there is no
        forked stitch pipeline), upload the reel to the bucket, and write a done-marker for the dispatcher's
        bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch requires."""
        from backend import orchestration
        from backend.config import StitchingConfig

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
        os.makedirs(scratch, exist_ok=True)
        # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to the bucket.
        config.output.output_dir = scratch
        # The dispatcher forwards storage.output_root via --set; --out-uri carries the same value as a
        # fallback so reel_object_uri (which reads storage.output_root) always resolves the reel's bucket
        # destination even if the override were dropped.
        if not (config.storage.output_root or "").strip():
            config.storage.output_root = args.out_uri

        spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
        video_id = str(spec.get("video_id") or "")
        out_name = str(spec.get("output_name") or "")
        source_ref = str(spec.get("video_uri") or "")
        locale = spec.get("locale")
        time_pairs = [
            (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
        ]
        stitching = StitchingConfig(**(spec.get("stitching") or {}))

        print(
            f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
            f"encoder={stitching.encoder} out={out_name}"
        )

        result = orchestration._stitch_reel(
            config,
            stitching,
            video_id,
            source_ref,
            time_pairs,
            out_name,
            locale,
            lambda stage: print(f"[worker] compile: {stage}"),
        )
        status = str(result.get("status") or "failed")
        rc = 0 if status == "success" else 3
        if status != "success":
            print(f"[worker] compile FAILED: {result.get('message')}")

        # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher passes it.
```

```
519         if getattr(args, "done_uri", None):
520             _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
521         return rc
522
523
524     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
525     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
526     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
527
528     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
```

31. assistant (2026-07-08T06:06:35.391Z)

Now let me see the worker's `main()`/dispatch to understand what happens if `cmd_compile` raises before writing a marker.

32. user (2026-07-08T06:06:39.578Z)

```
56:        except ValueError:
61:def _apply_overrides(config: Any, sets: List[str]) -> None:
170:    except StartupCheckError:
198:    except RemoteExecutionFailed as e:
203:    except PolicyTimeout as e:
240:    except Exception as e:  # never fail a good run because the marker push hiccuped
319:        except Exception as e:  # never mask the real failure on an artifact-push hiccup
460:    except Exception as e:  # never fail a good stitch because the marker push hiccuped
587:    except (
625:    except NotImplementedError as e:
970:def main(argv=None) -> int:
973:    return args.func(args)
976:if __name__ == "__main__":
977:    sys.exit(main())
```

33. user (2026-07-08T06:06:44.883Z)

```
960         # configured logging at import), which would leave the level at WARNING and re-
suppress the
961         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
962         logging.basicConfig(
963             level=logging.DEBUG if verbose else logging.INFO,
964             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
965             datefmt="%H:%M:%S",
966             force=True,
967         )
968
969
970     def main(argv=None) -> int:
971         args = build_parser().parse_args(argv)
972         _setup_logging(getattr(args, "verbose", False))
973         return args.func(args)
974
975
976     if __name__ == "__main__":
977         sys.exit(main())
978
```

34. assistant (2026-07-08T06:06:51.843Z)

The worker `main()` at line 973 calls `args.func(args)` with NO try/except. Let me compare how `cmd_infer` handles this ? does it always write a marker even on failure?

35. user (2026-07-08T06:06:53.489Z)

```
120         video_id: str,
121         segments: List[dict],
122         status: str,
123         storage_cfg: dict,
124         video_uri: str = "",
125     ) -> List[str]:
126         """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127         ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129         Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132         report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
134         out_local = os.path.join(scratch, "outputs", video_id)
135         os.makedirs(out_local, exist_ok=True)
136         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137         _write_report_json(
138             video_id,
139             segments,
140             status,
141             os.path.join(out_local, "report.json"),
142             video_uri=video_uri,
143         )
144         out_prefix = out_uri.rstrip("/")
145         pushed: List[str] = []
146         for fn in sorted(os.listdir(out_local)):
147             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149             pushed.append(uri)
150         return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         )  # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False  # already logged at ERROR by enforce_startup_checks
172
```

```
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180         from the local 2/3), never a raw traceback:
181           * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
182             running (the shim best-effort cancels it).
183           * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
184             crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185             ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186         from backend.compute import RemoteExecutionFailed
187         from backend.infrastructure.execution_policy import PolicyTimeout
188
189         spec = ComputeJobSpec(
190             video_uri=args.video_uri,
191             out_uri=args.out_uri,
192             segmenter=args.segmenter,
193             config_overrides=tuple(args.set or ()),
194             skip_validation=bool(getattr(args, "skip_validation", False)),
195         )
196         try:
197             result = compute.run_infer(spec)
198         except RemoteExecutionFailed as e:
199             logger.error(
200                 "[worker] remote execution terminated without a completion marker: %s", e
201             )
202             return 5
203         except PolicyTimeout as e:
204             logger.warning(
205                 "[worker] remote compute did not complete (no marker within budget): %s", e
206             )
207             return 4
208         logger.info(
209             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210             result.video_id,
211             result.returncode,
212             result.outputs_uri,
213         )
214         return result.returncode
215
216
217     def _write_done_marker(
218         done_uri: str,
219         video_id: str,
220         returncode: int,
221         segment_count: int,
222         status: str,
223         storage_cfg: dict,
224         scratch: str,
225     ) -> None:
226         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
227         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
228         try:
```

```
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(
            args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
        )
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
        # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
        # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
        # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
        # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
        # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
        # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
        # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
```

```
        # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
        if getattr(args, "skip_validation", False):
            print(
                "[worker] validation SKIPPED (--skip-validation): the dispatcher is the authoritative "
                "gate and already validated this input."
            )
            val_results, val_context = [], {}
        else:
            val_results, val_context = validator_registry.run_all_with_context(
                local_video, config
            )
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(
            video_id,
            local_video,
            "failed" if failures else "processed",
            [r.model_dump() for r in val_results],
        )

        def _fail(rc: int) -> int:
            # A worker-side FAILURE (validation, unknown segmenter, or a detector timeout/abort the
            # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success". Write + push
            # a status="failed" report.json/intervals.csv (0 segments) so the control plane's bucket
            # truth (submit-cache probe, restart-recovery) and the alpha-user result surface see an
            # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup must never
            # mask the original failure. THEN the M6 done-marker (real rc + status) so a remote
            # dispatcher's bucket-poll terminates FAST instead of hanging until its budget expires.
            try:
                _serialize_and_push_outputs(
                    scratch,
                    args.out_uri,
                    video_id,
                    [],
                    "failed",
                    storage_cfg,
                    str(getattr(args, "video_uri", "") or ""),
                )
            except Exception as e:  # never mask the real failure on an artifact-push hiccup
                logger.warning("[worker] could not push failure artifacts: %s", e)
            # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(
                    args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
                )
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
```

```
337                 )
338                 return _fail(2)
339             seg_name = args.segmenter or config.indexing.default_segmenter
340             segmenter_cls = segmenter_registry.get(seg_name)
341             if not segmenter_cls:
342                 print(
343                     f"[worker] unknown segmenter: {seg_name!r} "
344                     f"(have {list(segmenter_registry.list_available())})"
345                 )
346                 return _fail(2)
347             segmenter_actual = seg_name
348             result = segmenter_cls(db, config).process_video(
349                 video_id, local_video, max_frames=args.max_frames
350             )
351             segmentation_ran = True
352             if isinstance(result, Failure):
353                 print(f"[worker] segmenter FAILED: {result.message}")
354                 return _fail(3)
355             segments = result.value if hasattr(result, "value") else result
356             status = "success"
357             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
360             if not segments:
361                 detector_impl = (
362                     os.environ.get("RALLY_DETECTOR_IMPL")
363                     or getattr(config.indexing, "detector_impl", None)
364                     or "auto"
365                 )
366                 logger.warning(
367                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
368                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
369                     "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
```

36. assistant (2026-07-08T06:07:21.406Z)

Let me read the full `_maybe_dispatch_remote` (detect path) to see how a worker validation-failure rc surfaces to the user, and confirm the marker has no message.

37. user (2026-07-08T06:07:26.362Z)

```
110:    if report.get("output_dir"):
126:    cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as a
job failure
251:    ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
253:        return None
256:        return None
266:        return None
339:        return bool(report and report.get("status") == "success")
376:                return None
379:                return None
383:                return None
385:            if not report or report.get("status") != "success":
386:                return None
393:                video_path = str(report.get("video_uri") or "") or _first_object_under(
400:                return None
408:                return None
422:        return None
450:def _maybe_dispatch_remote(
483:                "status": "failed",
```

```
525:        return None  # explicit "run on my machine" ? in-process regardless of the server's
target
547:        return None  # default-OFF / placed on control_plane ? the in-process segmenter
below
688:    if result.returncode != 0:
689:        return _failed(f"cloud job failed (returncode {result.returncode}).", True)
751:def _execute_processing(
799:        raise
806:        raise
933:                "status": "failed",
952:                "status": "failed",
993:            raise
1001:                "status": "failed",
1043:        # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
1072:    Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat that
needs a
1087:        return None
1110:        raise _CompileError(404, "Indexed video record not found.")
1115:        raise _CompileError(400, str(e)) from e
1120:        raise _CompileError(400, "No matching play segments found to stitch.")
1141:        raise _CompileError(
1174:        return None
1178:    return None
1206:    ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
1232:        # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count (for
remote
1241:            "status": "failed",
1303:        return {"status": "failed", "message": e.detail, "video_id": video_id}
1354:            "status": "failed",
1387:            "status": "failed",
1391:    if result.returncode != 0:
1393:            result.report.get("message")
1394:            or f"cloud compile job failed (returncode {result.returncode})."
1396:        return {"status": "failed", "message": msg, "video_id": video_id}
1399:        result.report.get("download_url")
1411:        "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
1425:    the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
1454:        return {"status": "failed", "message": e.detail, "video_id": video_id}
```

38. user (2026-07-08T06:07:37.460Z)

```
676         try:
677             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
678             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
679             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
680             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
681             result = compute.run_infer(
682                 spec,
683                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
684             )
685         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
686             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
687             return _failed(f"cloud dispatch error: {e}", True)
688         if result.returncode != 0:
689             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
690         if result.video_id and result.video_id != video_id:
691             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
692             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
```

```
diagnosable.
            logger.warning(
                "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
                "and the worker hash; ingesting under the local id.",
                result.video_id,
                video_id,
            )

        notify("ingesting (cloud_run)")
        storage_dict = (
            storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
            if hasattr(storage_cfg, "model_dump")
            else {}
        )
        try:
            _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
        except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
            logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
            return _failed(
                f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
                True,
            )

        # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
        segments = [
            s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
        ]
        _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
        # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
        # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
        provenance = {
            "path": "cloud_run",
            "requested": req.compute,
            "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
            "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
            "execution": result.execution,
            "outputs_uri": result.outputs_uri,
        }
        logger.info(
            "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
            result.execution,
            provenance["job"],
            provenance["region"],
            result.outputs_uri,
            video_id,
        )
        return (
            {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
                "compute": provenance,
            },
            True,
        )
```

```
749
750
```

39. assistant (2026-07-08T06:09:36.553Z)

Let me read the top of `_maybe_dispatch_remote` (the actual detect decision) and `_execute_processing`'s validation-delegation block, to check `_will_detect_on_cloud` faithfully mirrors the real dispatch decision (a mismatch could create a nobody-validates or double-validate).

40. user (2026-07-08T06:09:39.495Z)

```
450     def _maybe_dispatch_remote(
451         cfg,
452         db,
453         req: "ProcessRequest",
454         video_id: str,
455         seg_name: str,
456         val_results,
457         play_wm: int,
458         cand_wm: int,
459         notify,
460     ) -> Optional[Tuple[Dict[str, Any], bool]]:
461         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
462         == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
463         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
464
465         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
466         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
467         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
468         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
469         the in-process path stays byte-identical)."""
470         from backend.compute import ComputeJobSpec, get_compute_backend
471
472         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
473             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
474             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
475             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
476             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
477             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
478             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
479             # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
480             return (
481                 {
482                     "video_id": video_id,
483                     "status": "failed",
484                     "message": msg,
485                     "results": [r.model_dump() for r in val_results],
486                     "play_segments": [],
487                     "compute": {
488                         "path": "cloud_run" if ran else "not_run",
489                         "requested": req.compute,
490                         "target": "cloud_run",
491                         "dispatched": ran,
```

```
492                     },
493                 },
494                 ran,
495             )
496
497         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
498         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
499         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
500         choice = (req.compute or "").strip().lower()
501         if choice and choice not in ("local", "cloud"):
502             logger.warning(
503                 "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
504                 req.compute,
505             )
506             choice = ""
507         if choice == "local":
508             # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
509             # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
510             # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
511             # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
512             # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
513             # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
514             # below exactly as before (the weights-present check is the escape hatch).
515             if (
516                 (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
517                 and _cloud_available(cfg)
518                 and not _local_detector_weights_present(cfg, seg_name)
519             ):
520                 return _failed(
521                     f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
522                     "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
523                     False,
524                 )
525             return None  # explicit "run on my machine" ? in-process regardless of the
server's target
526         if choice == "cloud":
527             if not _cloud_available(cfg):
528                 return _failed(
529                     "cloud-serving was requested but this server isn't configured for it "
```

41. assistant (2026-07-08T06:10:09.841Z)

Let me check the validation-delegation tests to see whether the worker-rejection message surfacing is covered, and read `_execute_processing`'s delegation block plus `_apply_overrides`.

42. user (2026-07-08T06:10:13.222Z)

```
31:    return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
43:    def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
45:        self.video_id, self.returncode = video_id, returncode
56:        if self.returncode == 0:
83:                    "returncode": self.returncode,
146:def test_cloud_dispatch_ingests_segments(cloud_env):
```

```
165:def test_cloud_dispatch_builds_correct_spec(cloud_env):
189:def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
202:def test_cloud_dispatch_forwards_wasb_serving_knobs(cloud_env):
222:def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
239:def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
240:    """The control-plane is the AUTHORITATIVE validation gate: it validated + only
dispatches on PASS
242:    redundant, decodes the video a second time, and ? because the control-plane's validation
config
244:    validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
dispatch MUST
245:    carry --skip-validation, telling the worker to trust the gate. Regression guard for that
wiring."""
252:    assert "--skip-validation" in fake.calls[0]
255:def test_cloud_failure_marks_failed_no_rows(cloud_env):
259:    fake = _FakeCloudClient(returncode=1)
266:    assert "returncode 1" in out["message"]
275:def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
285:    assert "cloud input" in out["message"]
289:def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path, monkeypatch):
330:def test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
367:def test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path,
monkeypatch):
380:    assert "cloud input" in out["message"]
384:def test_default_off_runs_in_process(tmp_path, monkeypatch):
422:    assert "cloud_run" not in out["message"]
425:def test_cloud_ingest_failure_marks_failed_no_leak(cloud_env):
442:                json.dump({"video_id": self.video_id, "returncode": 0}, f)
450:    assert "could not read the result" in out["message"]
454:def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
493:def test_per_request_local_forces_in_process(cloud_env):
510:    assert "cloud_run" not in out["message"]
516:def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
558:def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
583:    assert "isn't configured" in out["message"]
586:def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
605:def test_capabilities_reports_cloud_available(cloud_env):
617:def test_capabilities_cloud_unavailable_by_default(monkeypatch, tmp_path):
630:def test_full_api_process_to_query_via_cloud(cloud_env):
```

43. user (2026-07-08T06:10:14.592Z)

```
40      from backend.reporting import emit_indexer_report
41      from backend.results import Failure
42      from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
43      from backend.utils.hashing import compute_video_id
44
45      logger = logging.getLogger("backend.worker")
46
47
48      def _coerce(v: str) -> Any:
49          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
50          low = v.lower()
51          if low in ("true", "false"):
52              return low == "true"
53          for cast in (int, float):
54              try:
55                  return cast(v)
56              except ValueError:
57                  pass
58          return v
59
60
61      def _apply_overrides(config: Any, sets: List[str]) -> None:
```

```
62          """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63          for kv in sets or []:
64              key, sep, val = kv.partition("=")
65              if not sep:
66                  raise ValueError(f"--set expects k=v, got {kv!r}")
67              parts = key.split(".")
68              obj = config
69              for p in parts[:-1]:
70                  obj = getattr(obj, p)
71              setattr(obj, parts[-1], _coerce(val))
72
73
74      def _write_intervals_csv(segments: List[dict], path: str) -> None:
75          """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
76          artifact (same schema as the rally-annotator labels)."""
77          with open(path, "w", newline="") as f:
78              w = csv.writer(f)
79              w.writerow(["start", "end", "shot_count", "ending_reason"])
80              for s in segments:
81                  w.writerow(
82                      [
83                          f"{float(s.get('start_time', 0.0)):.3f}",
84                          f"{float(s.get('end_time', 0.0)):.3f}",
85                          s.get("shot_count", ""),
86                          s.get("ending_reason", s.get("shot_count_confidence", "")),
87                      ]
88                  )
89
90
91      def _write_report_json(
92          video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93      ) -> None:
94          with open(path, "w") as f:
95              json.dump(
96                  {
97                      "video_id": video_id,
98                      "status": status,
99                      "segment_count": len(segments),
```

44. user (2026-07-08T06:10:22.326Z)

```
1:@@ -1103,3 +1103,73 @@ def test_wait_progress_message_is_honest_and_minute_granular():
2:     assert "wasb_hybrid" in early and "starting up" in early
3:     later = _wait_progress_message("wasb_hybrid", 125.0)
4:     assert later.endswith("2 min elapsed")
7:+# --- #521: phase_placement.validation (delegate validation to the L4 worker)
------------------
10:+def test_validation_delegated_to_worker_when_placed_on_l4(cloud_env):
12:+    run its validator suite ? it delegates to the worker, which validates (dispatch carries
NO
22:+    cfg = AppConfig.model_validate(
37:+    assert out["status"] == "success"
38:+    assert ran["n"] == 0  # the control-plane skipped its validators (delegated)
39:+    assert "--skip-validation" not in fake.calls[0]  # so the worker validates instead
42:+def test_validation_not_delegated_when_detection_runs_in_process(cloud_env, monkeypatch):
44:+    runs in-process, no worker) ? the control-plane MUST still validate. Never leave a clip
unvalidated
55:+    cfg = AppConfig.model_validate(
72:+    assert out["status"] == "success"
73:+    assert ran["n"] == 1  # in-process ? the control-plane validated (no worker to delegate
to)
74:+    assert fake.calls == []  # ran locally, never dispatched
```

45. user (2026-07-08T06:10:37.248Z)

```
666:    worker_skips_validation = (
674:        skip_validation=worker_skips_validation,
689:        return _failed(f"cloud job failed (returncode {result.returncode}).", True)
861:        # 2-vCPU control-plane). The worker then validates (skip_validation=False) and
rejects a bad
865:        validate_on_control_plane = (
```

46. assistant (2026-07-08T06:11:57.878Z)

I now have enough to finalize. Let me do one last confirmation of the infer done-marker's field set (no message) versus the compile marker, to lock finding #1.

47. user (2026-07-08T06:12:00.655Z)

```
855             # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
856             # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
857             #
858             # #521 phase_placement.validation: when validation is placed on the L4 AND
detection will
859             # actually dispatch there, the control-plane DELEGATES validation to the worker
? it does NOT
860             # run the validator suite here (the whole point is to move the CPU-bound
validation OFF the
861             # 2-vCPU control-plane). The worker then validates (skip_validation=False) and
rejects a bad
862             # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed job.
When detection
863             # is NOT on the cloud (in-process), the control-plane always validates ? there
is no worker to
864             # delegate to (the config guard makes validation=cloud_run_l4 require a cloud
segment).
865             validate_on_control_plane = (
866                 resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
867                 or not _will_detect_on_cloud(config, getattr(req, "compute", None))
868             )
```

48. assistant (2026-07-08T06:12:11.649Z)

I have completed my robustness-security review of the #521 diff (branch `feat/521-compile-on-l4`, commit `8866ffe` ? note: this PR is NOT yet in the master working tree, so I reviewed the branch version; line numbers below are from that commit's post-change files).

Summary of what I verified as SAFE (no defect): the worker's `output_name` traversal ? even though `cmd_compile` trusts `spec["output_name"]` without re-running `safe_output_filename`, the control-plane validates it before staging, the local write jails it via `os.path.basename` (compiler.py:481), and the bucket mirror uses a flat object key, so it is not exploitable. Storage forwarding (`output_root`/`backend` via `--set`, with `--out-uri` fallback) correctly lets the worker read the spec and mirror the reel. The NVENC fallback is all-or-nothing (encoder resolved once before the clip loop). `_will_detect_on_cloud` faithfully mirrors the real dispatch decision, so the nobody-validates guard holds at runtime. Malformed-spec crashes degrade to failed jobs via the #515 crash-fast probe.

Two real defects found, reported below.

49. user (2026-07-08T06:12:31.226Z)

Structured output provided successfully